

Ejercicio 7: Mediana de dos vectores ordenados (Divide y Vencerás)

Enunciado

Dados dos vectores de enteros $X[1 \dots n]$ e $Y[1 \dots n]$, ordenados de forma creciente, diseña un algoritmo que calcule la mediana de los $2n$ elementos en tiempo $\mathcal{O}(\log n)$ y sin utilizar vectores adicionales.

1. Concepto y Estrategia de Divide y Vencerás

La mediana de un conjunto de $2n$ elementos ordenados (número par de elementos) es el **elemento inferior de la mediana** (el elemento en la posición n de los $2n$ elementos ordenados, usando indexación de 1 a $2n$).

En lugar de mezclar los dos vectores en un único vector de tamaño $2n$ (lo cual costaría $\mathcal{O}(n)$ en tiempo y espacio), aplicamos una estrategia de **Divide y Vencerás**:

1. Calculamos el punto medio de los subvectores actuales, $m = \lfloor n/2 \rfloor$.
2. Comparamos los elementos medianos de ambos subvectores: $X[x_i + m - 1]$ y $Y[y_i + m - 1]$.
3. **Descarte de mitades**:
 - Si $X[x_i + m - 1] \leq Y[y_i + m - 1]$: Significa que la mediana global no puede estar en la primera mitad de X ni en la segunda mitad de Y . Descartamos estas zonas y repetimos la búsqueda sobre la segunda mitad de X y la primera mitad de Y .
 - Si $X[x_i + m - 1] > Y[y_i + m - 1]$: Descartamos la segunda mitad de X y la primera mitad de Y .

Cada paso recursivo reduce el tamaño del problema a la mitad, garantizando un coste de $\mathcal{O}(\log n)$.

2. Explicación de los Casos Base (Según la Pizarra)

Cuando el tamaño del intervalo actual n llega a ser muy pequeño ($n = 1$ y $n = 2$), se aplican las siguientes reglas directas para evitar recursión infinita o desbordamiento de índices:

- **Caso $n = 1$** : Tenemos solo dos elementos: $X[x_i]$ y $Y[y_i]$. La mediana (el menor de los dos) es sencillamente:

$$\min(X[x_i], Y[y_i])$$

- **Caso $n = 2$** : Tenemos cuatro elementos en juego: $\{X[x_i], X[x_s]\}$ y $\{Y[y_i], Y[y_s]\}$. Queremos encontrar el segundo elemento más pequeño de los cuatro:

- **Caso 1** ($X[x_s] < Y[y_i]$): Al estar ordenados, el orden total es $X[x_i], X[x_s], Y[y_i], Y[y_s]$. El segundo elemento es $X[x_s]$.
- **Caso 2** ($Y[y_s] < X[x_i]$): Por simetría, el orden total es $Y[y_i], Y[y_s], X[x_i], X[x_s]$. El segundo elemento es $Y[y_s]$.
- **Caso 3 (Por defecto)**: Si hay solapamiento entre los intervalos, el segundo elemento más pequeño será el máximo de los dos elementos iniciales: $\max(X[x_i], Y[y_i])$.

3. Algoritmo Completo en Pseudocódigo

```
// Función recursiva principal
Función Mediana2V(X, Y, xi, xs, yi, ys):
    n = xs - xi + 1 // Tamaño del subproblema actual

    // CASO BASE 1: Solo queda un elemento en cada subvector
    Si (n == 1):
        Retornar Minimo(X[xi], Y[yi])

    // CASO BASE 2: Quedan dos elementos en cada subvector
    Si (n == 2):
        Si (X[xs] < Y[yi]):
            Retornar X[xs] // Caso 1 (X totalmente a la izquierda de Y)
        Si (Y[ys] < X[xi]):
            Retornar Y[ys] // Caso 2 (Y totalmente a la izquierda de X)
        Retornar Maximo(X[xi], Y[yi]) // Caso 3 (Intervalos solapados)

    // DIVIDE Y VENCERÁS (Reducción del tamaño a la mitad)
    m = parte_entera_inferior(n / 2)

    // Comparación de los elementos centrales de los subvectores
    Si (X[xi + m - 1] <= Y[yi + m - 1]):
        // Descartamos la primera mitad de X y la segunda mitad de Y
        Retornar Mediana2V(X, Y, xi + m, xs, yi, ys - m)
    Sino:
        // Descartamos la segunda mitad de X y la primera mitad de Y
        Retornar Mediana2V(X, Y, xi, xs - m, yi + m, ys)

// Función envoltura (Llamada inicial)
Función CalcularMediana(X, Y, n):
    Si (n == 0):
        Retornar "Error: Vectores vacíos"
    Retornar Mediana2V(X, Y, 1, n, 1, n)
```

4. Análisis de Complejidad

- **Complejidad Temporal:** $\mathcal{O}(\log n)$. En cada paso recursivo calculamos $m = \lfloor n/2 \rfloor$ y descartamos exactamente m elementos de un vector y m del otro. La ecuación de recurrencia del algoritmo es:

$$T(n) = T(n/2) + \mathcal{O}(1)$$